Assignment 5

DevOps and CI/CD: Automating Processes for Research Projects

Part 1. The role of DevOps and CI/CD in automating research projects (interpretable credit scoring research)

Part 2. Deploying a CI/CD pipeline for a Python project with Docker and Jenkins

Project: fuzzy-scorecard, https://github.com/Gontafi/fuzzy-scorecard

Alibek Asset, Astana IT University

October 2026

Contents

Part 1. The role of DevOps and CI/CD in automating research projects.....3
1. What is DevOps: goals, principles and benefits.....3
2. Overview of CI/CD.....3
3. The role of DevOps tools in research environments.....4
4. Examples of Docker, Jenkins and Kubernetes in scientific and engineering projects. .5
5. Conclusion: what DevOps solves in scientific teams.....5
Part 2. Practical task: CI/CD pipeline with Docker and Jenkins.....6
6. Initial setup.....6
7. Containerising the project.....6
8. Deploying the Jenkins server.....6
9. The pipeline.....7
10. Difficulties encountered.....7
11. Conclusions.....7
Appendix. Screenshots of the working Jenkins pipeline.....8
References.....11

Part 1. The role of DevOps and CI/CD in automating research projects

1. What is DevOps: goals, principles and benefits

DevOps is a way of organising software work in which the people who develop a system and the people who run it share one process, one set of tools and one responsibility for the result. The term joins “development” and “operations”, and it appeared around 2009 as a reaction to the usual conflict between the two groups: developers wanted to ship changes quickly, operations wanted stability and so resisted change. DevOps resolves the conflict by making change small, frequent, automated and measured, so that it stops being dangerous.

Kim, Humble, Debois and Willis (2016) summarise the principles as “three ways”. The first is flow: work moves from development to the user in small batches through an automated path, with no manual hand-offs and no long queues. The second is feedback: every change is checked as early as possible by tests, monitoring and reviews, so that a problem is found where it was made. The third is continual learning: the team experiments, keeps what works and treats failures as information. A popular checklist of the same ideas is CALMS: culture (shared responsibility), automation (of builds, tests, deployment and infrastructure), lean (small batches, limited work in progress), measurement (of lead time, failure rate, recovery time) and sharing (of knowledge and tools).

The benefits are documented in the State of DevOps research summarised by Forsgren, Humble and Kim (2018). Teams that practise continuous integration, automated testing, version control of everything, trunk-based development and infrastructure as code deploy more often, have shorter lead times from commit to production, recover faster from incidents and have a lower change failure rate; the four measures move together, so speed is not bought with stability. For a research group the same practices give a different but related benefit: an experiment that is built, tested and run by an automated pipeline can be repeated by anyone, at any time, with the same result.

2. Overview of CI/CD

Continuous integration (CI) is the practice of merging every developer’s work into the main line at least daily and verifying each merge with an automated build and test run (Fowler, 2006). Three rules make it work: the build is fully automated and starts from a clean machine, it finishes in minutes, and a broken build is fixed before anything else is merged. Continuous delivery (CD) extends the pipeline to a packaged, deployable artefact that can be released with one action, and continuous deployment removes even that action. Humble and Farley (2010) describe the whole chain as a deployment pipeline and recommend building the artefact once and promoting the same artefact through the stages.

A typical pipeline has stages for checkout, static analysis, unit tests, build of the artefact (a package or a container image), integration or smoke tests, and deployment. The pipeline is described as code in the repository (a Jenkinsfile, a GitHub Actions workflow, a GitLab CI file), triggered by every commit or pull request, and it reports its status next to the change. In Assignment 3 I built such a pipeline for the fuzzy-scorecard package on GitHub Actions; in this assignment the same project is built with Docker and Jenkins, which are the tools most often found in self-hosted research infrastructure.

3. The role of DevOps tools in research environments

Research software has a reproducibility problem that is well documented. Results depend on the exact versions of a language, of dozens of libraries and of system packages; the code runs on the author’s machine and nowhere else; the experiment is launched by hand in a notebook and nobody can say later which version of the code produced the figure in the paper. Wilson et al. (2014) list the consequences, and Boettiger (2015) argued that containers address the environment part of the problem directly: a Dockerfile is an executable description of the environment, and an image built from it runs identically on a laptop, a cluster node or a reviewer’s machine years later. Nüst et al. (2020) give practical rules for writing such Dockerfiles for data science, for example pinning versions, keeping the image small and running as an unprivileged user.

Three kinds of tools matter in a research environment. Containers (Docker, and on HPC clusters Singularity or Apptainer) freeze the environment. A CI server (Jenkins, GitHub Actions, GitLab CI) runs the tests and the experiment pipeline on every change on a clean machine, so the reproducibility of a result is checked each time the code changes. An orchestrator (Kubernetes) runs many containers across many machines: parameter sweeps, bootstrap replications, shared notebook servers and scoring services for pilots. Around them sit version control for code and configuration, artefact storage for images and results, and monitoring. The combination gives a research group what DevOps gives a product team: every result traceable to a commit, an image and a pipeline run.

In my own research, a fuzzy rule-based mortgage credit scorecard trained on 1.4 million Freddie Mac loans, the needs are concrete. The evaluation runs twenty bootstrap refits of several models and must give the same numbers to the two papers and to the thesis; a reviewer of the papers must be able to run the public part of the code; a bank model-risk team must be able to audit the implementation. A container image of the package, a pipeline that tests every change and a smoke run that writes a JSON report as an artefact answer all three.

4. Examples of Docker, Jenkins and Kubernetes in scientific and engineering projects

Docker. The BioContainers project maintains thousands of Docker and Singularity images of bioinformatics tools so that a pipeline can name an exact tool version instead of asking the user to compile it (da Veiga Leprevost et al., 2017). The Rocker project provides versioned R images used for reproducible statistical analyses, and the Jupyter Docker Stacks provide ready notebook environments for data science. In this project the Dockerfile builds two targets from one definition: the runtime image of the package and a test image that adds pytest and ruff, so the tests run against the same bits that are shipped.

Jenkins. Jenkins is the oldest widely used open-source CI server and remains common in institutes and companies that host their own infrastructure because it runs on a single machine, has plugins for almost every tool and describes pipelines as code in a Jenkinsfile. Large open-source engineering projects such as those of the Apache Software Foundation run their builds on Jenkins, and CERN's ROOT data analysis framework used Jenkins for its continuous integration across platforms for many years. In this assignment a Jenkins controller runs in a container, polls GitHub every two minutes and executes the six-stage pipeline described in Part 2.

Kubernetes. Kubernetes is the orchestrator that grew out of Google's Borg system (Burns et al., 2016). CERN runs Kubernetes clusters on its OpenStack cloud for physics analysis services and batch work; the Pangeo community runs geoscience analysis on Kubernetes with JupyterHub and Dask, so that a scientist opens a notebook and gets a cluster of workers on demand; the "Zero to JupyterHub" project is the standard way to give a class or a research group a shared notebook server. For a credit scoring project Kubernetes would be the natural home of a scoring API in a bank pilot and of parallel bootstrap jobs; it was not needed for the scale of this assignment, so the deployment stage targets the local Docker host instead.

5. Conclusion: what DevOps solves in scientific teams

DevOps practices address the weaknesses of scientific software listed in Section 3. When the environment is a Dockerfile in the repository, the code runs the same way on every machine, and the phrase "it works on my machine" loses its meaning. A pipeline that refuses to build an image whose tests fail keeps untested code out of the results. Because the pipeline documents how the software is built, tested and run, anyone with access to the repository can repeat an experiment, which reduces the dependence on the one person who wrote it, and the same artefact comes out of the same commit every time. Breaking changes in libraries surface within minutes of a commit; without a pipeline they tend to appear months later, during a revision. Hand-over to a user, in my case a bank, becomes a container image and a pipeline run. Part 2 shows what this costs for a small package: a Dockerfile, a Jenkinsfile and about a day of configuration.

Part 2. Practical task: CI/CD pipeline with Docker and Jenkins

6. Initial setup

The sample research project is the Python package fuzzyscore from Assignment 3: a monotone fuzzy scorecard for credit risk with 22 unit tests, a command-line demo on synthetic loans and a GitHub Actions workflow. The code is at https://github.com/Gontafi/fuzzy-scorecard. For this assignment the repository received a `Dockerfile`, a `Jenkinsfile` and a folder `deploy/jenkins` with the definition of the Jenkins server itself, so that the whole setup can be reproduced from the repository. Jenkins was used as the task requires; the GitHub Actions workflow from Assignment 3 remains in place and runs in parallel on GitHub.

7. Containerising the project

The Dockerfile has two stages built from python:3.12-slim. The base stage copies the package metadata and sources, installs the package with pip, creates an unprivileged user and sets the entry point to the fuzzyscore command, so that docker run fuzzyscore demo runs the experiment. The test stage starts from base, adds the tests directory, installs pytest, pytest-cov and ruff, sets COVERAGE_FILE to a writable location and PYTHONPATH to the source tree, and sets the entry point to pytest. The runtime image is 327 MB; the full file is in the repository.

8. Deploying the Jenkins server

Jenkins runs locally in a container built from the official jenkins/jenkins:lts-jdk21 image (Jenkins 2.580.1). The custom image adds the Docker command-line client, so that pipeline stages can build and run images through the host's Docker socket, which is mounted into the container; the jenkins user is added to the group that owns the socket. Plugins are installed at build time from plugins.txt (pipeline, git, docker-workflow, junit, pipeline-stage-view, configuration-as-code, job-dsl, matrix-auth and others). The server is configured entirely by a Configuration-as-Code file: it creates the admin user from an environment variable, gives anonymous users read access so that the pipeline pages can be viewed without login, and defines a multibranch pipeline job that points at the GitHub repository and rescans it every two minutes. The setup wizard is disabled, so the server is ready a minute after docker run, and the whole definition is in the repository under deploy/jenkins.

```
docker build --build-arg DOCKER_GID=$(stat -c %g /var/run/docker.sock) -t
jenkins-a5 deploy/jenkins
docker run -d --name jenkins-a5 -p 8081:8080 -v
jenkins-a5-home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock -e JENKINS_ADMIN_PASSWORD=...
jenkins-a5
```

9. The pipeline

The Jenkinsfile is a declarative pipeline with six stages after checkout (Figure 3). Build test image builds the test target of the Dockerfile and tags it with the build number. Lint runs ruff check and ruff format inside that image. Unit tests runs pytest inside the image with a JUnit report and coverage, copies the reports out of the finished container with docker cp and publishes them to Jenkins with the junit step, so that every build shows its test results and trend. Build image builds the runtime target and tags it with the build number and latest. Smoke test runs fuzzyscore demo in the new image and archives the JSON report as a build artefact. Deploy to test server, which runs only on the main branch, replaces the container fuzzyscore-test on the Docker host with the new image; this is the optional continuous delivery part, with the Docker host playing the role of the test server. A post action removes the test image. The pipeline is triggered by polling the repository every two minutes (a webhook cannot reach a laptop), so every commit on GitHub is built within minutes.

The first run (build 1) failed in the unit test stage and the second run, after a fix to the Jenkinsfile, passed in 33 seconds; the stage view, the build summary, the test report, the console log and the deployed container are shown in Figures 1 to 7.

10. Difficulties encountered

Four problems took most of the time. First, Configuration as Code failed at start-up twice because plugins named in the configuration were missing (matrix-auth, job-dsl); the fix is one line in plugins.txt. Second, the first pipeline mounted the workspace into the test container with a bind mount, as one would on a laptop. The Docker daemon lives on the host, where the Jenkins workspace path does not exist, so it created an empty root-owned directory and pytest could not write its reports; the same mount had silently let the lint stage pass on an empty directory. The fix was to drop bind mounts: lint and tests run inside the test image and docker cp copies the reports out. Third, inside the container the coverage file was not writable by the unprivileged user and coverage measured the installed copy instead of the source tree; COVERAGE_FILE, PYTHONPATH and a chown solved this. Fourth, the Docker client in the Jenkins image uses the legacy builder and prints a deprecation warning; installing buildx is a follow-up.

11. Conclusions

The project is now built, tested, packaged, smoke-tested and deployed to a test container by Jenkins on every commit, with the server itself defined as code. Compared with GitHub Actions in Assignment 3, Jenkins needed more setup (an image, plugins, configuration as code, a socket mount) but gives full control of the infrastructure, which a bank or an institute without cloud access needs. The exercise showed the gap between a laptop and a pipeline: three of the four difficulties

were about paths, users and permissions that never appear on a developer's machine, and the pipeline found all of them on the first run.

Links: repository https://github.com/Gontafi/fuzzy-scorecard; Jenkinsfile https://github.com/Gontafi/fuzzy-scorecard/blob/main/Jenkinsfile; Dockerfile https://github.com/Gontafi/fuzzy-scorecard/blob/main/Dockerfile; Jenkins server definition https://github.com/Gontafi/fuzzy-scorecard/tree/main/deploy/jenkins; GitHub Actions runs https://github.com/Gontafi/fuzzy-scorecard/actions.

Appendix. Screenshots of the working Jenkins pipeline

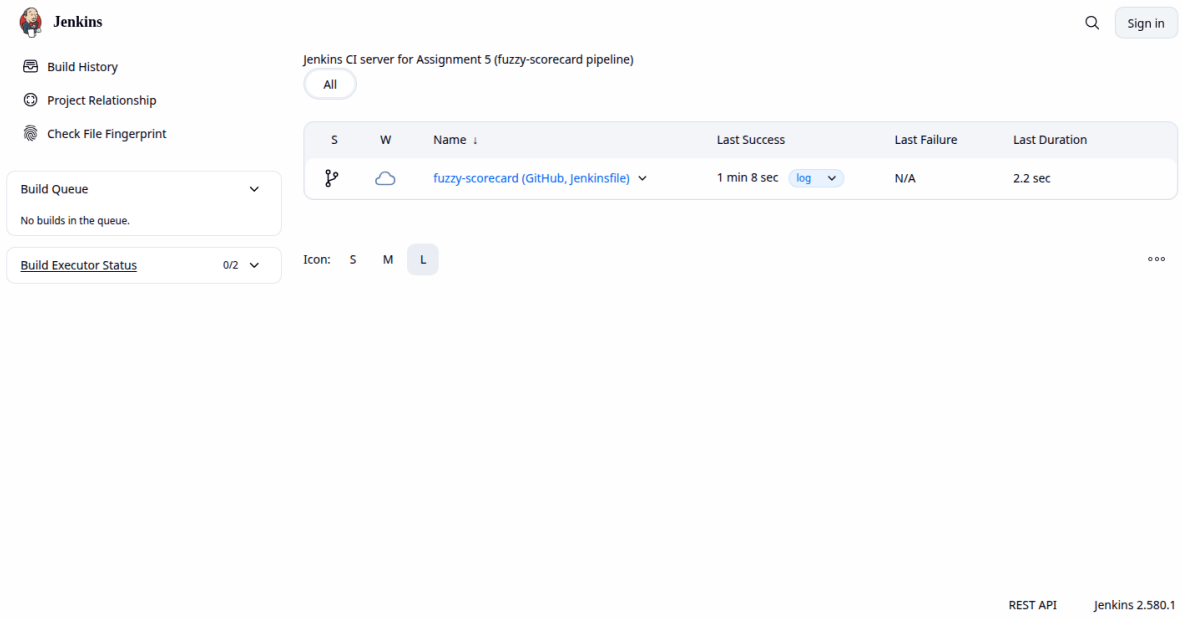


Figure 1. Jenkins dashboard with the multibranch job fuzzy-scorecard (Jenkins 2.580.1, configured by code).

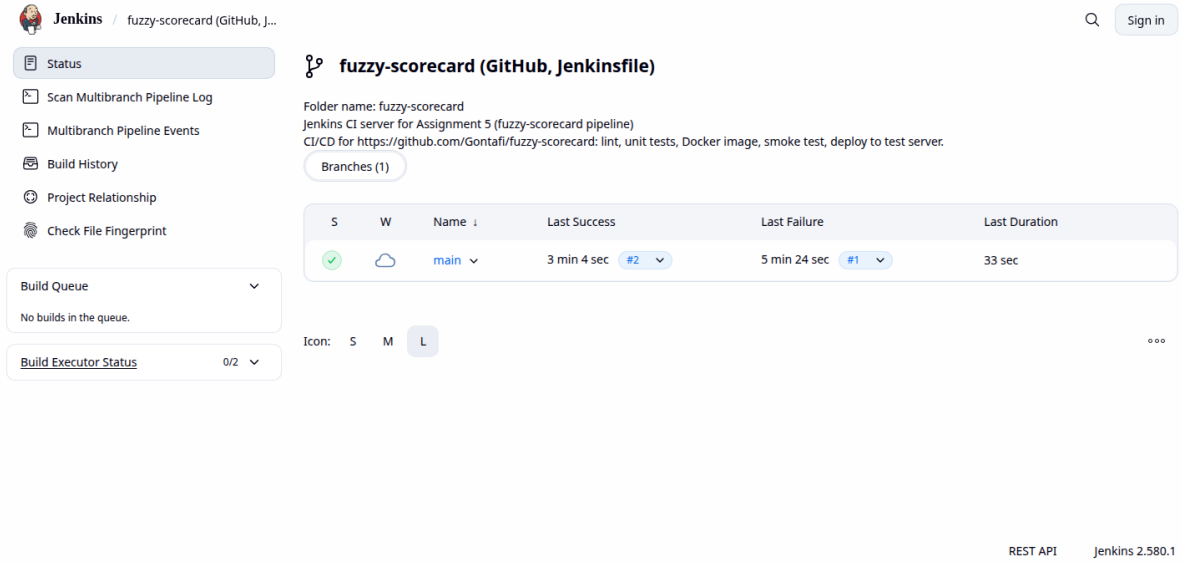


Figure 2. The multibranch job: the main branch with its build history (build 1 failed, build 2 succeeded).

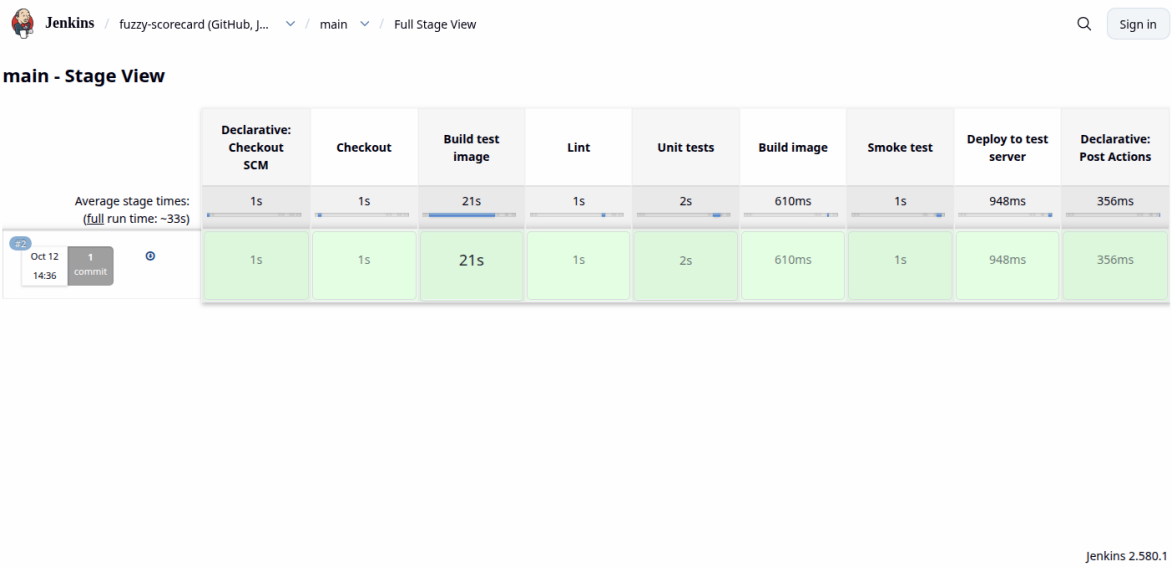


Figure 3. Stage view of build 2: checkout, build test image, lint, unit tests, build image, smoke test, deploy to test server; 33 seconds in total.

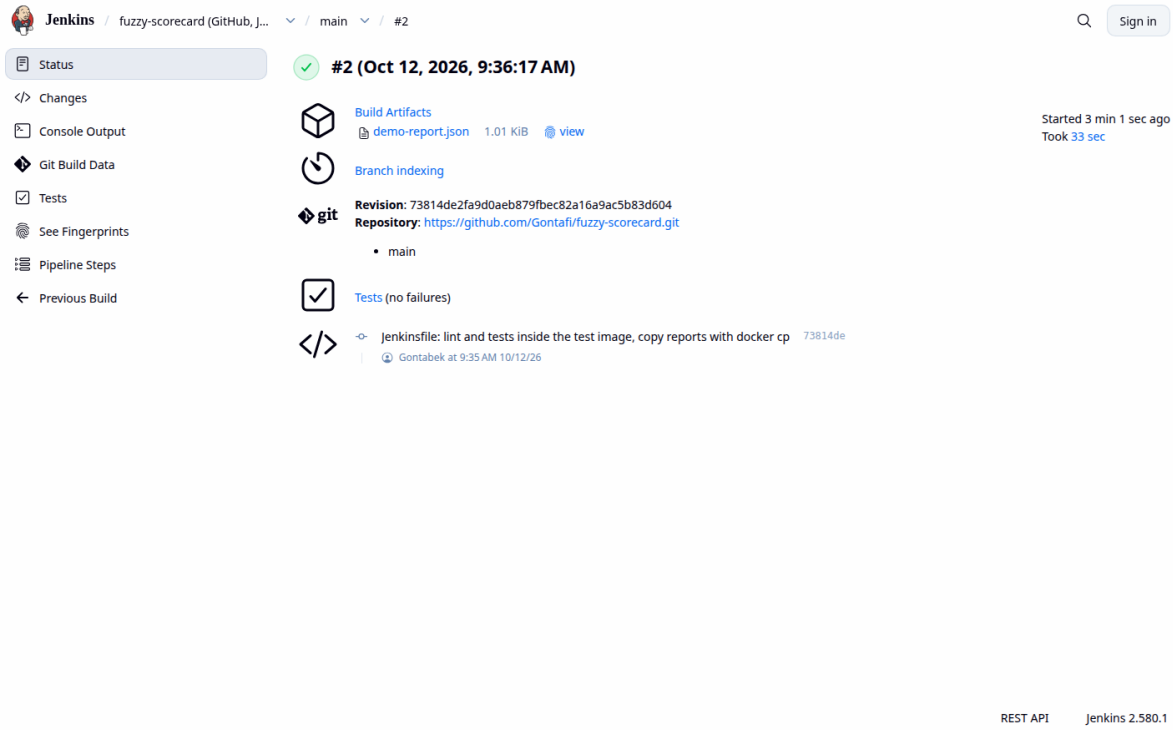


Figure 4. Build 2 summary: commit, repository, artefact demo-report.json and test results.

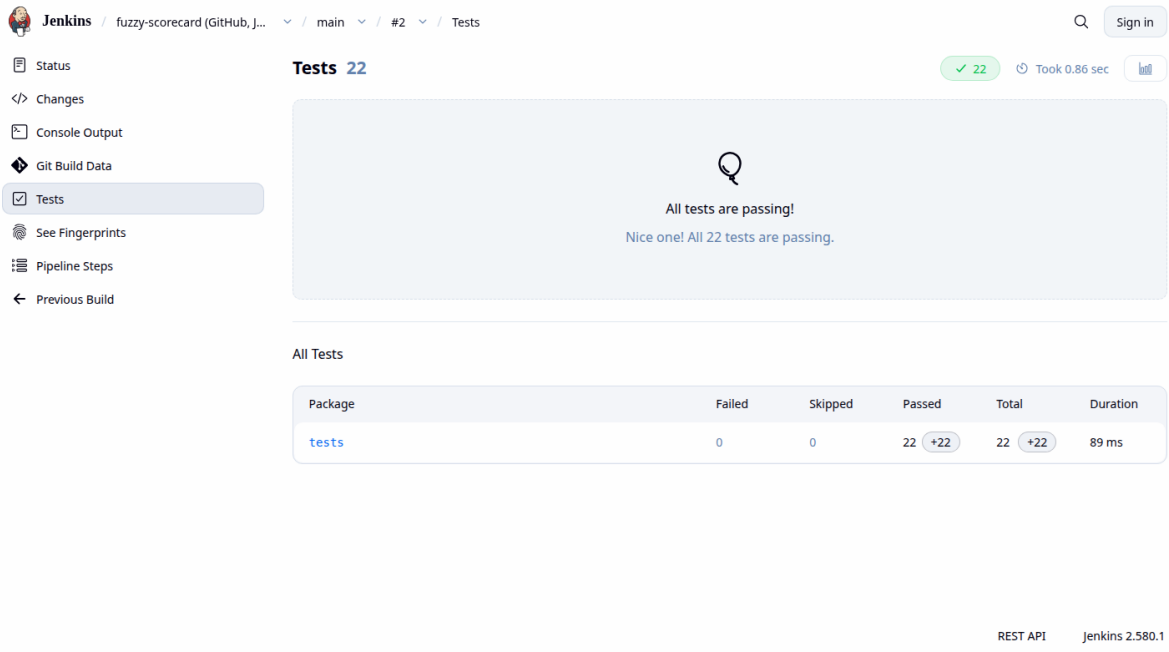


Figure 5. JUnit test report published by the pipeline: 22 tests passed.

```
[2026-10-12T09:36:49.394Z]   "brier": 0.032427719526785505,
[2026-10-12T09:36:49.394Z]   "ece": 0.00397564278752231,
[2026-10-12T09:36:49.394Z]   "active_terms": 11,
[2026-10-12T09:36:49.394Z]   "scorecard": [
[2026-10-12T09:36:49.394Z]     {
[2026-10-12T09:36:49.394Z]       "term": "fico is at most 760",
[Pipeline] archiveArtifacts
[2026-10-12T09:36:49.430Z] Archiving artifacts
[2026-10-12T09:36:49.434Z] Recording fingerprints
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy to test server)
[Pipeline] sh
[2026-10-12T09:36:49.852Z] + docker rm -f fuzzyscore-test
[Pipeline] sh
[2026-10-12T09:36:50.140Z] + docker run -d --name fuzzyscore-test --restart unless-stopped --entrypoint sh fuzzyscore:2 -c while true; do fuzzyscore d
[2026-10-12T09:36:50.140Z] 1c8c1e43cc8dfb06aed7a26df49b90ef32893fadb589569659f09e1d48091dcf
[Pipeline] sh
[2026-10-12T09:36:50.439Z] + docker ps --filter name=fuzzyscore-test
[2026-10-12T09:36:50.439Z] CONTAINER ID   IMAGE          COMMAND                  CREATED         STATUS                  PORTS     NAMES
[2026-10-12T09:36:50.439Z] 1c8c1e43cc8d   fuzzyscore:2   "sh -c 'while true; …"   1 second ago   Up Less than a second             fuzzyscore-test
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] sh
[2026-10-12T09:36:50.783Z] + docker rmi -f fuzzyscore-test:2
[Pipeline] echo
[2026-10-12T09:36:50.820Z] Image fuzzyscore:2 built, tested and deployed.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // timestamps
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Figure 6. End of the console log of build 2: image build, smoke test, deployment and the final SUCCESS.

```
$ docker ps --filter name=fuzzyscore-test
NAMES             IMAGE          STATUS         COMMAND
fuzzyscore-test   fuzzyscore:2   Up 5 minutes   "sh -c 'while true; …"

REPOSITORY   TAG       SIZE      CREATED
fuzzyscore   2         327MB     5 minutes ago
fuzzyscore   latest    327MB     5 minutes ago
fuzzyscore   local     327MB     18 minutes ago

$ docker exec fuzzyscore-test head -9 /tmp/last.json
{
  "version": "0.1.0",
  "n_train": 2000,
  "n_test": 2000,
  "default_rate": 0.0335,
  "auc": 0.7750152496699122,
  "brier": 0.030600447724123216,
  "ece": 0.0023583464003918424,
  "active_terms": 9,
```

Figure 7. The Docker host after the deployment stage: the fuzzyscore-test container runs image fuzzyscore:2 and writes its periodic demo report.

References

Boettiger, C. (2015). An introduction to Docker for reproducible research. ACM SIGOPS Operating Systems Review, 49(1), 71–79.

Burns, B., Grant, B., Oppenheimer, D., Brewer, E. and Wilkes, J. (2016). Borg, Omega, and Kubernetes. Communications of the ACM, 59(5), 50–57.

da Veiga Leprevost, F. et al. (2017). BioContainers: an open-source and community-driven framework for software standardization. Bioinformatics, 33(16), 2580–2582.

Forsgren, N., Humble, J. and Kim, G. (2018). Accelerate: The Science of Lean Software and DevOps. IT Revolution Press.

Fowler, M. (2006). Continuous Integration. martinfowler.com/articles/continuousIntegration.html.

Humble, J. and Farley, D. (2010). Continuous Delivery. Addison-Wesley.

Kim, G., Humble, J., Debois, P. and Willis, J. (2016). The DevOps Handbook. IT Revolution Press.

Nüst, D., Sochat, V., Marwick, B., Eglen, S. J., Head, T., Hirst, T. and Evans, B. D. (2020). Ten simple rules for writing Dockerfiles for reproducible data science. PLoS Computational Biology, 16(11), e1008316.

Wilson, G. et al. (2014). Best practices for scientific computing. PLoS Biology, 12(1), e1001745.

Jenkins project. Pipeline syntax and Configuration as Code plugin documentation. https://www.jenkins.io/doc/

Asset, A. (2026). fuzzy-scorecard repository, https://github.com/Gontafi/fuzzy-scorecard.